22 preguntas y respuestas

Dos ejercicios por tema T01–T11, con justificación y criterio de validación. El anexo original no fue adjuntado; esta propuesta cubre la estructura solicitada.

Ejercicio 01 — T01: Funciones puras en JavaScript

Pregunta: Escribir una función que calcule cuántas tareas están completadas sin modificar el arreglo.

Respuesta: const contar = (tareas) => tareas.filter(t => t.status === "completada").length;

Justificación: filter crea un arreglo nuevo y la función no cambia el arreglo de entrada.

Criterio de validación: Con 3 tareas, dos completadas, devuelve 2; el arreglo original permanece idéntico.

Ejercicio 02 — T01: Promesas y async/await

Pregunta: Explicar cómo obtener tareas de la API y tratar los errores de red.

Respuesta: async function listar(){ const res=await fetch("/api/tasks"); if(!res.ok) throw new Error("HTTP " + res.status); return (await res.json()).data; }

Justificación: await espera la promesa; comprobar res.ok evita tratar un error HTTP como respuesta válida.

Criterio de validación: Con 200 retorna array; con 401/500 lanza error; un catch de la vista muestra mensaje.

Ejercicio 03 — T02: Interfaces reutilizables

Pregunta: Definir una interfaz TypeScript para una tarea con ID, título y estado.

Respuesta: interface Tarea { id: string; title: string; status: "pendiente" | "en_progreso" | "completada"; }

Justificación: Los tipos literales impiden asignar estados no admitidos durante la compilación.

Criterio de validación: Asignar status: "invalido" produce error de TypeScript; una tarea válida compila.

Ejercicio 04 — T02: Genéricos y enumeraciones

Pregunta: Definir un resultado de API genérico y un enum para los estados.

Respuesta: enum Estado { Pendiente="pendiente", Progreso="en_progreso", Completada="completada" }; interface ApiResultado<T> { data:T; message?:string }

Justificación: ApiResultado<T> se reutiliza para usuarios, tareas y estadísticas manteniendo tipos coherentes.

Criterio de validación: ApiResultado<Tarea[]> solo admite un arreglo de tareas en data.

Ejercicio 05 — T03: Componentes reutilizables

Pregunta: Indicar cómo organizar dashboard, lista y formulario en Angular.

Respuesta: Declarar DashboardComponent, TasksComponent y TaskFormComponent en AppModule; navegar mediante RouterModule.forRoot(routes).

Justificación: Separar componentes reduce duplicación y hace más simple mantener cada función.

Criterio de validación: Las rutas /, /tareas y /tareas/nueva muestran sus componentes sin recargar la página.

Ejercicio 06 — T03: Data binding y directivas

Pregunta: Mostrar el título de una tarea y repetir una lista con Angular.

Respuesta: <li *ngFor="let t of tareas">{{ t.title }}

Justificación: *ngFor itera sobre los datos del componente y {{ }} usa interpolación de datos.

Criterio de validación: Una lista de 3 elementos genera 3 filas; si cambia el arreglo se actualiza la vista.

Ejercicio 07 — T04: Formulario reactivo

Pregunta: Escribir una validación Angular para exigir título entre 3 y 120 caracteres.

Respuesta: title: ["", [Validators.required, Validators.minLength(3), Validators.maxLength(120)]]

Justificación: Los formularios reactivos validan antes de enviar y permiten mostrar errores por campo.

Criterio de validación: Los títulos vacíos, de 2 o 121 caracteres invalidan el formulario.

Ejercicio 08 — T04: HttpClient e interceptores

Pregunta: ¿Cómo adjuntar un token JWT en solicitudes protegidas de Angular?

Respuesta: En un HttpInterceptor: req.clone({setHeaders:{Authorization:`Bearer ${token}`}}), luego next.handle(reqClonada).

Justificación: La cabecera centralizada evita repetir código en cada servicio HttpClient.

Criterio de validación: Inspeccionar petición a /api/tasks y verificar Authorization: Bearer; sin token debe devolver 401.

Ejercicio 09 — T05: Rutas REST

Pregunta: Definir las cuatro rutas HTTP para CRUD en Express.

Respuesta: POST /api/tasks crea; GET /api/tasks lista; PUT /api/tasks/:id actualiza; DELETE /api/tasks/:id elimina.

Justificación: Los verbos HTTP representan operaciones distintas sobre el mismo recurso.

Criterio de validación: Probar respuestas 201, 200, 200 y 200 y revisar que los cambios persistan.

Ejercicio 10 — T05: Manejo de errores

Pregunta: ¿Qué debe responder el backend si un ID de tarea no tiene formato MongoDB válido?

Respuesta: HTTP 400 con JSON {"message":"Identificador inválido"}. No enviar traza de error al cliente.

Justificación: Los errores de entrada pertenecen al cliente, y un contrato consistente facilita su tratamiento.

Criterio de validación: Solicitar GET /api/tasks/no-es-objectid y comprobar 400 con contenido JSON.

Ejercicio 11 — T06: Modelado Mongoose

Pregunta: Crear una propiedad title obligatoria y con límite de longitud en un Schema.

Respuesta: title: {type:String,required:true,trim:true,minlength:3,maxlength:120}

Justificación: MongoDB persiste los documentos; Mongoose realiza validación de modelo antes de guardar.

Criterio de validación: Insertar título de 2 caracteres desde la API es rechazado; uno válido permanece tras reiniciar.

Ejercicio 12 — T06: Actualizaciones persistentes

Pregunta: ¿Por qué utilizar runValidators:true al actualizar un documento?

Respuesta: Task.findOneAndUpdate(filtro, {$set:datos}, {new:true, runValidators:true});

Justificación: En Mongoose algunos validadores de actualización se encuentran desactivados por defecto.

Criterio de validación: Intentar actualizar título con dos caracteres falla; otro valor válido sí se almacena.

Ejercicio 13 — T07: Contraseñas con bcrypt

Pregunta: Describir registro y validación segura de contraseñas.

Respuesta: Registro: hash=await bcrypt.hash(password,12). Login: await bcrypt.compare(password,user.passwordHash).

Justificación: No se necesita guardar ni transmitir a la base la contraseña en texto claro.

Criterio de validación: MongoDB contiene passwordHash y no password; una contraseña errónea devuelve 401.

Ejercicio 14 — T07: JWT y control de roles

Pregunta: ¿Cómo impedir que un usuario común acceda a GET /api/admin/users?

Respuesta: Usar requireAuth y requireRole(Role.ADMIN) en backend antes del controlador.

Justificación: Una protección de interfaz no basta: el servidor valida JWT y rol almacenado en MongoDB.

Criterio de validación: Sin token: 401; token user: 403; token admin válido: 200.

Ejercicio 15 — T08: Refactorización

Pregunta: Explicar una mejora al separar rutas, controladores y modelos.

Respuesta: Pasar consultas Mongoose a una capa clara; dejar en rutas solo autenticación, validación y coordinación.

Justificación: Disminuye el acoplamiento y facilita pruebas y cambios sin alterar otras pantallas.

Criterio de validación: Conservar las respuestas de CRUD y documentar archivos modificados en CHANGELOG.md.

Ejercicio 16 — T08: Bitácora de cambios

Pregunta: Registrar una corrección de validación en la bitácora.

Respuesta: 2026-11-09: se añadió runValidators:true en PUT/PATCH para impedir títulos fuera de rango.

Justificación: Una bitácora registra qué se corrigió y el motivo del cambio.

Criterio de validación: Existe entrada fechada con causa, archivo/área afectada y resultado comprobable.

Ejercicio 17 — T09: React Hooks

Pregunta: Crear un estado local para la lista de tareas usando React.

Respuesta: const [tasks,setTasks]=useState<Task[]>([]); useEffect(()=>{api.tasks().then(r=>setTasks(r.data));},[]);

Justificación: useState guarda datos del componente; useEffect dispara la carga cuando se monta.

Criterio de validación: La lista aparece tras la respuesta real y muestra error si la API no está disponible.

Ejercicio 18 — T09: React Router DOM

Pregunta: Definir una ruta protegida al formulario en React Router.

Respuesta: <Route element={<Guard/>}><Route path="tareas/nueva" element={<TaskForm/>}/></Route>

Justificación: El componente Guard consulta el usuario autenticado y redirige al login cuando falta.

Criterio de validación: Sin sesión, ir a /tareas/nueva redirige a /ingresar; con sesión abre el formulario.

Ejercicio 19 — T10: Context API

Pregunta: ¿Qué información conviene almacenar en Context API para la sesión?

Respuesta: user, loading, signIn(), signUp(), signOut(), expuestos por AuthProvider y useAuth().

Justificación: La sesión se necesita en navbar, rutas y panel de administración; Context evita pasar props por toda la jerarquía.

Criterio de validación: Después de login todos los componentes consumidores muestran el mismo usuario.

Ejercicio 20 — T10: Actualización inmutable

Pregunta: ¿Cómo reemplazar una tarea actualizada sin alterar el arreglo previo de React?

Respuesta: setTasks(prev => prev.map(t => t.id===editada.id ? editada : t));

Justificación: map produce un arreglo nuevo y permite a React detectar el cambio de referencia.

Criterio de validación: Solo cambia el objeto con id correspondiente y prev no se modifica.

Ejercicio 21 — T11: Expiración de tokens

Pregunta: ¿Qué hacer cuando Fetch recibe HTTP 401 con sesión antes válida?

Respuesta: Eliminar token de sessionStorage, limpiar el contexto user y redirigir al formulario de login.

Justificación: El token puede caducar o ser revocado; mantenerlo en memoria genera un estado falso de autenticación.

Criterio de validación: Con token expirado, GET /api/tasks devuelve 401 y el panel vuelve a /ingresar.

Ejercicio 22 — T11: Cierre de sesión y API

Pregunta: Describir por qué un logout de backend es preferible a borrar el token solo del navegador.

Respuesta: POST /api/auth/logout incrementa tokenVersion del usuario y después el cliente limpia sessionStorage.

Justificación: La revocación del lado servidor invalida tokens emitidos con la versión anterior incluso si alguien conservó una copia.

Criterio de validación: Tras logout, reutilizar el token antiguo en GET /api/auth/me devuelve 401.